

PIXEL SOFTWARE DESIGN

Département de Recherche & Développement

Intelligence Artificielle Avancée

& Federated Learning

- Fondations de l'intelligence artificielle
- Algorithmes d'apprentissage fédéré
- Confidentialité, attaques et défenses
- Applications et grands modèles fédérés
- Framework FedX de référence
- Nouveaux fronts de recherche
- Travaux pratiques, projets et évaluation

Auteur	**Ghiloufi Jamel**
Fonction	**Architecte Logiciel Principal**
Direction	**Département de Recherche & Développement**
Service	**R&D - Intelligence Artificielle Avancée & Apprentissage Fédéré**
Niveau	**Recherche**
Édition	**Édition 2026 – 1.0**
Copyright	**© Pixel Software Design**

Ce document est la propriété exclusive de Pixel Software Design. Toute reproduction, représentation, adaptation ou diffusion, totale ou partielle, par quelque procédé que ce soit et sur quelque support que ce soit, est interdite sans autorisation écrite préalable de l'auteur.

Copyright et mentions légales

Éditeur	Pixel Software Design
Auteur	Ghiloufi Jamel
Fonction	Architecte Logiciel Principal
Département	Département de Recherche & Développement
Intitulé	Intelligence Artificielle Avancée & Federated Learning
Sous-titre	Fondations mathématiques de l'apprentissage automatique, algorithmes fédérés, protection de la vie privée, défense contre les attaques, grands modèles de langage fédérés, systèmes embarqués, renforcement appris, applications critiques (santé, V2X), plateforme de référence FedX et feuille de route de recherche.
Édition	Édition 2026 (version 1.0)
Langue	Français
Niveau	Recherche
Collection	Supports de formation et de recherche, Pixel Software Design

Droits d'auteur

© 2026 Pixel Software Design. Tous droits réservés.

Ce document est la propriété exclusive de Pixel Software Design. Toute reproduction, représentation, adaptation ou diffusion, totale ou partielle, par quelque procédé que ce soit et sur quelque support que ce soit, est interdite sans autorisation écrite préalable de l'auteur.

Droits de l'auteur. Ghiloufi Jamel est l'auteur unique de l'ensemble du contenu scientifique, pédagogique et technique de cet ouvrage. Toute adaptation, traduction, reproduction ou exploitation, y compris partielle, requiert une autorisation écrite préalable.

Droits des tiers. Les algorithmes, modèles et travaux cités restent la propriété de leurs auteurs respectifs. Les références employées figurent en fin d'ouvrage, ce qui permet de remonter au travail original pour toute affirmation chiffrée.

Marques et licences externes. PyTorch, TensorFlow, Flower, Opacus, MONAI, scikit-learn et PySyft sont des logiciels libres sous licences respectives (BSD, Apache 2.0, MIT). Leur usage dans ce document est purement pédagogique et n'emporte aucune prescription commerciale.

Mentions relatives aux données et aux expérimentations

Document pédagogique à usage de formation interne. Les exemples de code fournis sont livrés à titre pédagogique ; les résultats numériques illustrés sont issus d'expérimentations conduites dans un cadre de recherche et ne constituent pas des performances de production.

Ce support synthétise les travaux de recherche du Département de Recherche & Développement de Pixel Software Design sur l'intelligence artificielle distribuée, l'apprentissage fédéré, la protection de la vie privée et les systèmes embarqués intelligents.

Sommaire

Structure détaillée du cours

Partie 1 — Fondements théoriques de l'intelligence artificielle 8

1.1 Logique symbolique, langages formels et limites du consensus 9

1.2 Théorie des langages de programmation et calcul différentiable 11

1.3 Compilation des graphes de calcul : dialectes et mémoire 13

1.4 Informatique théorique : complexité, information, graphes 15

1.5 Science des données et méthode empirique 18

1.6 Les disciplines qui préparent l'IA de demain 19

Annexes 22

A.1 Liste des abréviations 23

A.2 Table de notation 23

Partie	Thème	Contenu	Compétence visée	Durée
Partie 1	Fondations	Statistique, optimisation, courbes d'apprentissage	Comprendre et formaliser	2-3 semaines
Partie 2	Algorithmes FL	FedAvg, convergence, hétérogénéité, système	Implémenter et régler	3-4 semaines
Partie 3	Confidentialité	DP, attaques, défenses, agrégation sécurisée	Auditer et sécuriser	2-3 semaines
Partie 4	Applications	FedLLM, santé, V2X, FedRL, TinyML, graphes	Transposer à un domaine	3-4 semaines
Partie 5	Framework FedX	Architecture, API, benchmarks, exploitation	Industrialiser	2 semaines
Partie 6	Nouveaux fronts	FedCL, FedCI, SNN, FedGL, feuille de route	Ouvrir la recherche	2 semaines
Partie 7	TP et projets	9 labs, 5 projets, corrigés, évaluation	Faire et rendre	4-6 semaines

Préface

Comment utiliser ce cours

Ce support est conçu pour un parcours de formation et de recherche couvrant l'ensemble de la chaîne de valeur de l'intelligence artificielle distribuée : les fondations mathématiques, les algorithmes d'apprentissage fédéré, les menaces et défenses, les applications de pointe, une plateforme de référence opérationnelle, et enfin les nouveaux fronts ouverts par la recherche du Département.

Ce n'est pas un manuel de référence à lire d'un trait : c'est un **manuel opératoire**. Chaque chapitre alterne exposition, formalisation, mise en œuvre numérique et évaluation. Les listings sont complets et exécutables ; les tableaux comparatifs servent de grille de décision ; les schémas d'architecture servent de support de revue de conception.

Public et prérequis

Le contenu s'adresse à des étudiants de master et de doctorat, à des ingénieurs IA et à des praticiens de la recherche appliquée. Les prérequis sont la connaissance de l'apprentissage automatique classique, de l'algorithmique numérique et des bases de probabilités. Les parties 1 à 3 sont auto-suffisantes pour un étudiant qui découvre le domaine ; les parties 4 à 7 supposent la mécanique et les outils présentés dans la partie 2.

Conventions de lecture. Une expression mathématique est encadrée par deux signes dollars ; un identifiant, un fichier ou une commande est écrit en caractères à chasse fixe ; les encadrés *Définition*, *Théorème*, *Méthode* et *Attention* signalent les points à maîtriser et les pièges fréquents. Les renvois de la forme « voir chapitre 3.2 » désignent le second chapitre de la troisième partie.

Progression pédagogique

Logique, langages, compilation, complexité et méthode

Partie 1

Fondements théoriques de l'intelligence artificielle

On peut utiliser l'intelligence artificielle sans connaître ses fondements. On ne peut pas en concevoir de nouvelles architectures, ni comprendre pourquoi une optimisation marche, sans eux. Ce cours assemble les six disciplines théoriques qui déterminent ce qui est possible, ce qui est rapide, et ce qui reste hors de portée.

Objectifs de la partie

- Lire un algorithme d'inférence formelle et connaître la limite de complétude du premier ordre.
- Relier une fonction de perte à une grandeur de la théorie de l'information.
- Expliquer pourquoi un réseau profond est un graphe de calcul, et ce qu'est sa descente.
- Justifier le passage d'une grammaire à une autre, et connaître la hiérarchie de Chomsky.
- Comprendre ce qu'apporte un compilateur de tenseurs, et pourquoi la fusion de noyaux change tout.
- Énoncer ce que le théorème de l'impossibilité du consensus distributed impose à un système asynchrone.

Plan de la partie

1. Logique symbolique, langages formels et limites du consensus
2. Théorie des langages de programmation et calcul différentiable
3. Compilation des graphes de calcul : dialectes et mémoire
4. Informatique théorique : complexité, information, graphes
5. Science des données et méthode empirique
6. Les disciplines qui préparent l'IA de demain

Chapitre 1.1

Logique symbolique, langages formels et limites du consensus

Trois fondations : ce qu'on sait énoncer, ce qu'on sait calculer, ce qu'il est impossible de garantir.

Langages formels et grammaires

Un langage formel est défini par un alphabet fini et un ensemble de règles de production. La grammaire décrit la syntaxe ; elle ne dit rien du sens des phrases.

$$G : S \to a\ S\ b \mid a$$
$$a\ S\ b \Rightarrow^{*} a^{n}\ b^{n}\ n \geq 1 \qquad \text{(F.1)}$$

La récursion produit des langages infinis à partir d'un nombre fini de règles. C'est tout le problème des langages de programmation : on décrit l'infini par le fini.

Type	Langage reconnu par	Capacité de mémoire	Exemple
Type 0	machine de Turing	aucune borneproblèmes indécidables	
Type 1	automate à pile	pile non bornée	expressions régulières, analyse syntaxique
Type 2	automate à pile	pile non bornée, transduction	la plupart des langages de programmation
Type 3	automate fini	un seul état courant	expressions régulières

Tableau 1 – La hiérarchie de Chomsky et son usage en intelligence artificielle

Le type 1 et le type 2 partagent le même pouvoir de mémoire ; ils diffèrent par la nature de l'algorithme de reconnaissance, pas par ce qu'ils décrivent.

Logique des prédicats et incomplétude

La logique des prédicats du premier ordre est expressive au point d'être incomplète : il existe des énoncés vrais dans toute structure sans être démontrables. C'est le théorème de Gödel, et c'est la raison pour laquelle une base de connaissances en logique des prédicats ne peut pas être complète.

$$\text{Satisfiabilité} = \exists\ M\ M \text{ models } \varphi$$
$$\text{Validité} = \forall\ M\ M \text{ models } \varphi \qquad \text{(F.2)}$$

Vérifier la satisfiabilité d'une formule du premier ordre est indécidable en général. En pratique, on utilise des solveurs qui explorent l'espace de structures de façon systématique et s'arrêtent au premier modèle.

> **La différence entre satisfiable et valide**
> Un énoncé peut être satisfiable sans être valide : il existe une structure où il est vrai, mais pas toutes. C'est exactement l'écart entre « il existe un patient qui réagit au traitement » et « le traitement fonctionne ». Confondre les deux est l'erreur logique la plus répandue dans les systèmes de règles.

Élément	Rôle	Exemple
Fait	élément connu, supposé vrai	`homme(socrate)`
Règle	condition déclenchante	`homme(X) :- mortel(X)`
Clause	faux et tête avec corps	`¬homme(X) v mortel(X)`
Question	ce que l'on cherche à déduire	`? mortel(socrate)`
Unification	cherche les variables compatibles	`X = socrate`
Résolution	combine des clauses pour en déduire une nouvelle	applique la clause 2 avec la négation correspondante

Tableau 2 – Programmation logique : de la clause a l'inférence

Le théorème de l'impossibilité du consensus

Dans un système asynchrone où un seul nœud peut tomber en panne, aucun algorithme déterministe ne peut garantir qu'un ensemble de nœuds réagissent à un même message. Ce résultat, connu par les initiales de ses auteurs, borne tout ce qu'on peut construire.

Definition – Théorème d'impossibilité

Il n'existe pas d'algorithme déterministe tolérant à une panne, fonctionnant dans un système asynchrone, qui garantisse l'accord d'un ensemble d'agents sur une valeur commune. La conclusion est qu'au moins une des propriétés suivantes doit être abandonnée : terminaison, sûreté, tolérance à la panne ou déterminisme.

Propriété	Signification	Ce qu'on fait concrètement
Accord exact	tous les nœuds ont la même valeur	tolérance
Temps borné	la décision arrive en temps fini	horloge ou délai
Tolérance à la panne	un nœud mort n'arrête pas le système	redondance et vote
Déterminisme	le calcul est reproductible	hasard ou calcul probabiliste

Tableau 3 – Ce que l'impossibilité impose en pratique

Aucun système réel ne satisfait les quatre. L'agencement pratique est presque toujours : accord exact, délai borné, tolérance à la panne, et hasard dans le choix du arbitre.

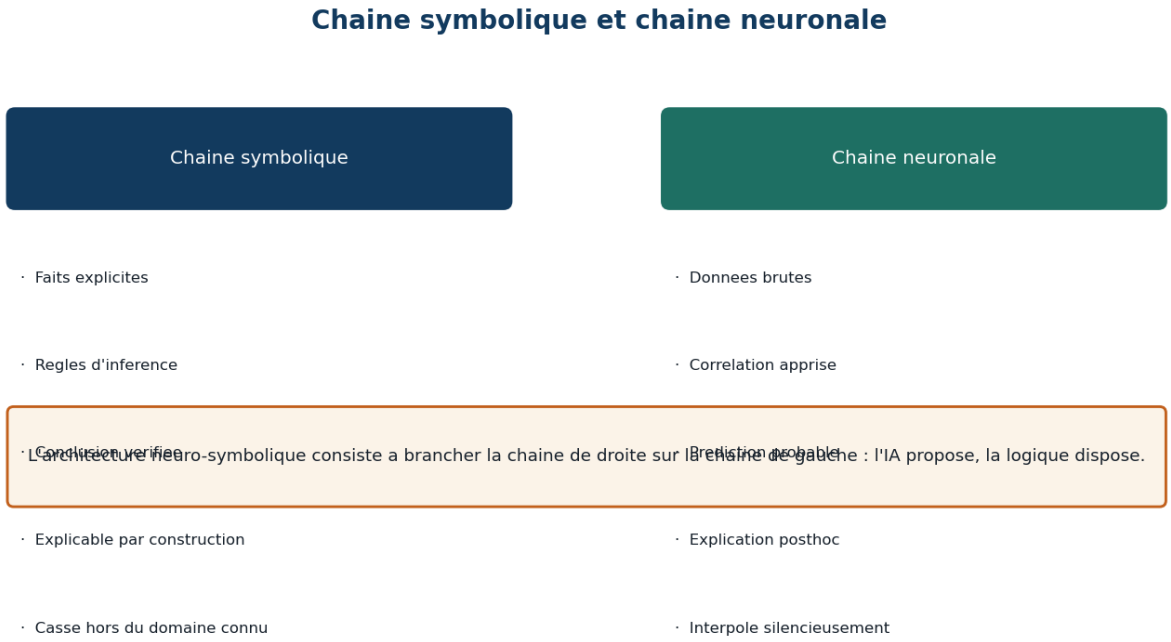


Figure 1 – Chaîne symbolique et chaîne neuronale : l'articulation neuro-symbolique

A retenir

- Un langage formel est infini, décrit par un ensemble fini de règles.
- La logique des prédicats est expressive mais incomplète : certaines vérités ne sont pas démontrables.
- Le théorème d'impossibilité borne l'accord exact, le temps borné, la tolérance à la panne et le déterminisme.
- Tout système réel abandonné au moins une de ces quatre propriétés.

Chapitre 1.2

Théorie des langages de programmation et calcul différentiable

Un réseau de neurones est un programme. C'est un programme qui se dérive.

Le lambda-calcul : modèle universel du calcul

Le lambda-calcul est un langage minimal — trois formes seulement — mais il est universellement puissant. Toute fonction calculable par une machine de Turing s'écrit dans le lambda-calcul, et réciproquement.

$$\text{overlinet} = \lambda\ x.\ t$$

$$(t \equiv t') \rightarrow (T \equiv \lambda\ x.\ U) \wedge (T' \equiv t'\ [\ x \mapsto t\ ]) \qquad \text{(F.3)}$$

L'application se lit comme une fonction : $T' = t'[x \mapsto t]$ signifie que l'argument est substitué dans le corps avant évaluation. Toute la sémantique d'un langage fonctionnel tient dans cette règle.

Caractéristique	Programmation fonctionnelle	Réseau de neurones
Fonction	valeur de premier ordre	couche = transformation
État	absent ou explicite	caches, mémoire, compteurs
Effet de bord	évité par construction	allocation mémoire
Répétition	récursion et composition	parallélisme par lots
Erreur	exhaustive, types stricts	silencieuse, dynamique

Tableau 1 – Programmation fonctionnelle et réseaux de neurones

Le lambda-calcul explique la forme du code, pas son efficacité. L'immutabilité des tenseurs sert un autre objet : la sûreté de l'exécution parallèle.

Systèmes de types et dimensions des tenseurs

L'erreur la plus fréquente en apprentissage profond n'est pas algorithmique : c'est l'incompatibilité de dimensions, détectée à l'exécution après des minutes de calcul sur accélérateur.

$$(d_1, d_2) \times (d_1, d_3) \to (d_2, d_3)$$
$$\text{incompatible} : (d_1, d_2) \times (d_3, d_4) \text{ si } d_2 \neq d_3 \qquad \text{(F.4)}$$

C'est la règle du produit matriciel, vérifiée statiquement. L'affichage explicite des dimensions dans les messages d'erreur est l'un des gains de productivité les plus élevés d'un outillage de vérification.

Quatre dimensions à vérifier systematiquement

- **Après toute normalisation** : la norme divide par un écart-type, il faut que ce dernier ne soit pas nul.

2. **Avant toute réduction de dimension** : le nombre de composantes ne peut pas dépasser le nombre d'exemples.
3. **Entre deux lots** : un lot vide ne doit pas produire un tenseur de dimension nulle.
4. **Avant toute réduction de mémoire** : la taille visée doit être alignée sur la granularité réelle des données.

Le calcul différentiable comme langage

Un programme devient apprenable lorsque chaque opération est différentiable. Le compilateur ne se contente plus à traduire : il construit un second programme, le graphe de rétropropagation, qui calculé les dérivées par rapport à chaque paramètre.

Algorithme 1 – Descente de gradient sur un réseau profond

```
01  Initialiser les poids au hasard, avec une graine fixéeChoisir un lot de données et passer la passe avant
02  Enregistrer chaque opération dans un graphe
03  Calculer la perte sur le lot, puis la derivee locale de chaque nœud
04  Propager vers l'arriere en accumulant les contributions
05  Mettre a jour chaque poids dans la direction opposee a sa derivee
06  Répéter jusqu'a epuisement du budget de calcul
07  Évaluer sur un jeu de données de validation, jamais sur celui d'entraînement
```

Entrées : Données étiquetées, architecture, budget d'itérations • **Sorties :** Poids ajustés, métriques de validation • **Complexité :** Coût de la passe arrière $\approx$ 2 fois la passe avant

La ligne 3 est la charnière : sans enregistrement, la rétropropagation est impossible. Elle est aussi la source de la plus grande consommation mémoire d'un entraînement.

A retenir

- Le lambda-calcul est un modèle universel du calcul, pas seulement de la programmation fonctionnelle.
- Une erreur de dimensions est une erreur de type déguisée.
- Le calcul différentiable transforme un programme en un second programme qui le dérive.

- L'enregistrement du graphe est la condition de la rétropropagation, et la première source de coût mémoire.

Chapitre 1.3

Compilation des graphes de calcul : dialectes et mémoire

Le même code doit tourner sur un processeur, une carte graphique ou un circuit. C'est un problème de langages.

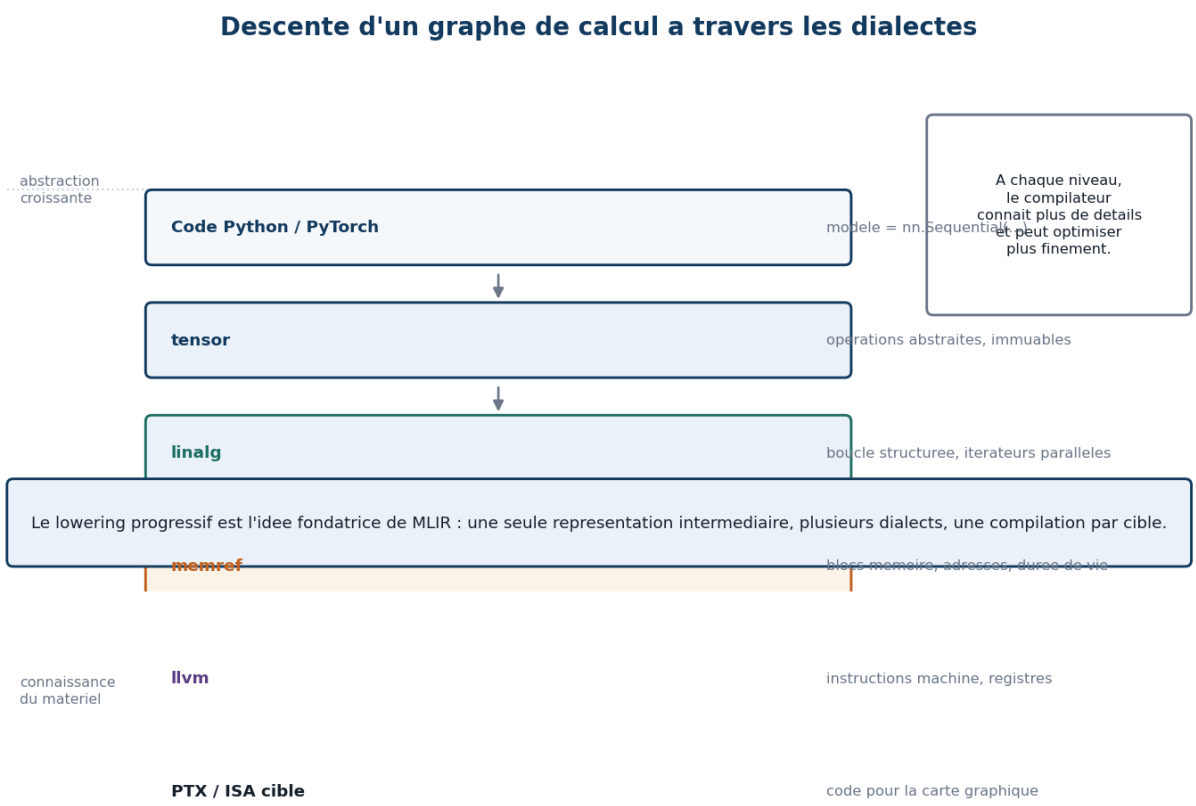


Figure 1 – Descente d'un graphe de calcul à travers les dialectes

Pourquoi une représentation intermédiaire unique ne suffit pas

Un compilateur classique travaille sur une représentation intermédiaire unique, proche de l'assembleur. Optimiser un réseau de neurones avec une telle représentation est presque impossible : à ce niveau, la structure mathématique du calcul a été perdue.

Dialecte

Un sous-ensemble de types, d'opérations et de règles de transformation, correspondant à un niveau d'abstraction précis. Un même programme peut être exprimé dans plusieurs dialectes, et le compilateur passe de l'un à l'autre par des transformations appelées *abaissements*.

Dialecte	Représente	Apport à l'optimisation
tensor	opérations mathématiques abstraitesfusion d'opérations sur tenseurs entiers	
linalg	itérations d'algèbre linéaire structurées	parallelisation, pavage, vectorisation
sparse	matrices creuses et leurs motifs	détection de structure, insertion de zéros
memref	blocs mémoire, adresses, durées de vie	allocation, réutilisation, vectorisation mémoire
scf	contrôle de flux structuré	débouclage, fusion de boucles, parallélisation
gpu	unités de calcul de la carte	occupation, registres, accès mémoire partagé
llvm	instruction machine générique	transformation finale

Tableau 1 – Les dialectes de la chaîne de compilation

Chaque dialecte ajoute une connaissance du matériel ou de l'algèbre. C'est cette connaissance accumulée qui permet d'optimiser là où une représentation unique ne le peut pas.

Exemple concret : l'addition de deux vecteurs

Prenons l'opération la plus simple et suivons-la à travers la chaîne. L'exemple est volontairement trivial : l'intérêt est dans la transformation, pas dans le calcul.

Algorithme 2 – Descente d'une addition élémentaire

```
01  Au niveau tensor : addition de deux tenseurs de 1024 éléments, sans étatConversion vers
linalg : une boucle générique, iterateur de type parallel
02  Conversion vers memref : des blocs mémoire, des adresses, une durée de vie
03  Analyse de durée de vie : le résultat est écrit puis plus jamais relu
04  Allocation : réutilisation du tampon d'entrée si les durées de vie ne se croisent pas
05  Vectorisation : regroupement des éléments par registres de la carte
06  Fission de boucle : distance de réutilisation comparée a la taille du tampon partage
07  Passage final : émission du code pour la carte cible
```

Entrées : Deux tenseurs de 1024 flottants de 32 bits • **Sorties :** Un tenseur de sortie, et le code machine correspondant • **Complexité :** Le gain vient de la suppression des allers-retours mémoire

L'étape 6 est celle qui change tout : sans elle, chaque élément passe par la mémoire globale, qui est le facteur limitant réel d'un accélérateur.

Figure indisponible

La figure `thm_mémoire` n'est pas encore implementee.

Les trois techniques de gestion de la mémoire

- **Analyse de durée de vie :** le compilateur détermine à quel instant un tenseur est créé, utilisé pour la dernière fois, puis mort. Cette information autorise la libération anticipée.
- **Réutilisation de tampons :** deux tenseurs dont les durées de vie ne se chevauchent pas peuvent occuper le même bloc physique. C'est la technique qui fait passer la mémoire d'un gigaoctet à deux cents mégaoctets dans un cas réel.
- **Fusion de noyaux :** si un résultat temporaire est produit puis relu immédiatement, il n'est jamais écrit en mémoire globale. Le calcul fusionné s'exécute dans les registres et la mémoire partagée du bloc.

La bande passante, pas le calcul

Un accélérateur passe l'essentiel de son temps à attendre des données, pas à calculer. Multiplier la puissance de calcul sans réduire les transferts mémoire ne produit aucune accélération. C'est pourquoi la fusion de noyaux est souvent plus rentable qu'une amélioration de l'algorithme lui-même.

Phase	Entrée	Sortie	Exemple d'optimisation
Capture	code source	graphe de haut niveauélagage des constantes, fusion d'opérations	
Intermédiaire	graphe	graphe structuré	vectorisation, pavage des boucles
Spécialisée	graphe structuré	graphe mémoire	allocation, réutilisation de tampons
Codegen	graphe mémoire	instruction machine	choix d'instructions, registres

Tableau 2 – Les quatre phases de la compilation d'un modèle

La phase critique est la troisième : c'est là que la mémoire est décidée, et c'est le facteur limitant.

A retenir

- Un dialecte est un niveau d'abstraction avec ses propres types et ses propres règles.
- L'abaissement progressif permet d'optimiser là où une représentation unique ne le peut pas.
- La durée de vie d'un tenseur est l'information qui autorisé la réutilisation de tampon.
- La fusion de noyaux supprime les allers-retours en mémoire globale, premier poste de coût.

Chapitre 1.4

Informatique théorique : complexité, information, graphes

Trois théories qui disent ce qui est faisable, ce qui est rapide, et ce que vaut une information.

Complexité : ce qui est hors de portée

L'informatique théorique classe les problèmes selon le temps et l'espace nécessaires pour les résoudre en fonction de la taille de l'entrée. La classe NP décrit les problèmes dont on vérifie facilement une solution proposée.

Figure 1 — Temps de calcul selon la taille du problème

$$\exists \text{ algorithme déterministe } A : A(x) \text{ en } O(2^{|x|}) \text{ ? Longrightarrow P stackrel?= NP} \quad (1.4.1)$$

La question est ouverte depuis des décennies. Sa réponse déterminerait la façon dont on conçoit toute une catégorie de systèmes : vérification, planification, cryptographie.

Classe	Problèmes	Conséquence pour l'IA
P	trouver une solution en temps polynomialalgorithms utilisables directement	
NP-complet	vérifier une solution est polynomial	on accepte une solution approximative
PSPACE	problèmes de jeux et de systèmes d'agents	recherche exhaustive nécessaire pour l'optimalité
Indécidable	aucun algorithme, par construction	impossibilité de vérifier toutes les propriétés

Tableau 1 – Les classes de complexité et leur place en intelligence artificielle

L'apprentissage automatique contourne la difficulté NP par statistique : il n'essaie pas de trouver la solution optimale, il apprend une politique bonne en moyenne.

Théorie de l'information

La théorie de l'information quantifie l'incertitude d'une distribution de probabilité. Elle fournit la fonction de perte de la quasi-totalité des modèles.

$$H(p) = - \sum_k p_k \log_2 p_k$$
$$L = - \sum_k p^*_k \log_2 \hat{p}_k \quad (F.5)$$

L'entropie mesure l'incertitude de la distribution vraie ; l'entropie croisée mesure à quel point la prédiction s'en écarte. Minimiser l'entropie croisée revient à maximiser la vraisemblance des données observées.

Quantité	Définition	Usage
Entropie	incertitude moyenne d'une distributionmesure de la qualité d'une prédiction	
Entropie croisée	écart entre distributions prédites et vraies	fonction de perte standard
Information mutuelle	information partagée par deux variables	séparation de composantes, réduction de dimension
Divergence de Kullback-Leibler	distance non symétrique	régression, ré-entraînement d'un modèle
Bande passante	capacité d'un canal de transmission	coût de communication d'un système distribué

Tableau 2 – Les quantités de la théorie de l'information en IA

Théorie des graphes

Un réseau de neurones profond est, mathématiquement, un graphe orienté acyclique de calcul : chaque nœud est une opération, chaque arête transporte un tenseur. Cette lecture explique la propagation de bout en bout et la rétropropagation.

- **Graphe orienté acyclique de calcul :** la structure d'un réseau profond ; le sens de parcours est l'ordre d'exécution.
- **Graphe de passage avant et arrière :** les deux sens de la rétropropagation sont deux parcours du même graphe.
- **Processus décisionnel de Markov :** le modèle formel d'un agent qui évolue entre états, formalisé par un triplet (états, actions, transitions).
- **Graphe de connaissance :** entités reliées par des relations typées, base d'un raisonnement symbolique explicable.

A retenir

- La classe NP borne ce qu'une recherche exhaustive peut produire ; l'apprentissage automatique la contourne par approximation.
- L'entropie croisée est la traduction directe de la théorie de l'information en fonction de perte.
- Un réseau profond est un graphe orienté acyclique de calcul, parcouru dans deux sens.
- Le processus de Markov est le modèle formel d'un agent qui évolue entre états.

Chapitre 1.5

Science des données et méthode empirique

La science des données est l'ensemble des contraintes qui empêchent une bonne méthode de produire un mauvais résultat.

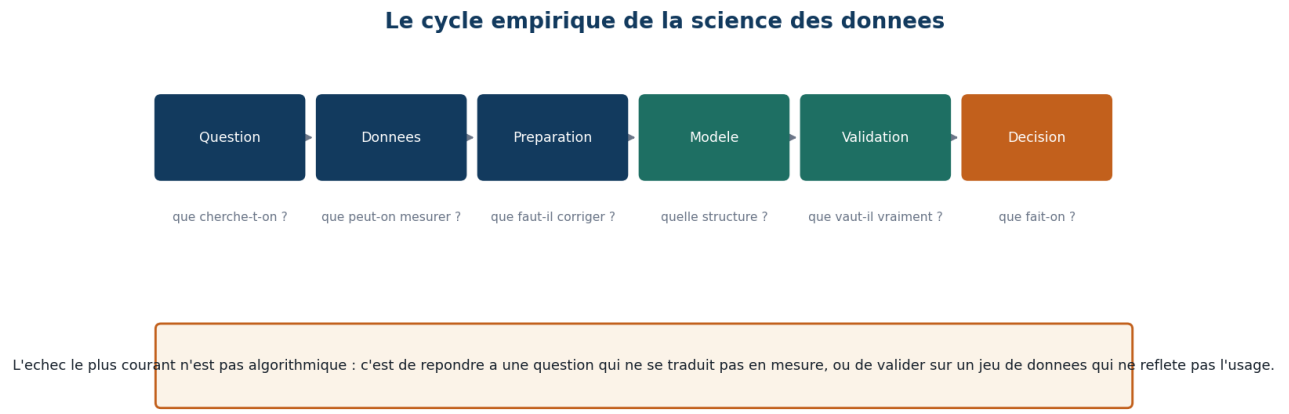


Figure 1 – Le cycle empirique de la science des données

Préparation : ce qui se joue avant le modèle

- **Valeurs manquantes :** comprendre pourquoi elles manquent. Une valeur absente après un refus de réponse est une information, pas une absence.
- **Normalisation :** indispensable dès que des variables ont des unités différentes, sans quoi la plus ample écrase toutes les autres.
- **Détection d'anomalies :** un point atypique n'est pas forcément une erreur, mais il mérite une explication.
- **Fuite d'information :** une variable qui n'existait pas au moment de la prédiction ne peut pas servir.

Statistiques et probabilités

Un modèle d'IA prédictive ne raisonne pas en certitudes mais en probabilités. La statistique fournit le vocabulaire pour quantifier cette incertitude et pour distinguer une amélioration réelle d'une fluctuation d'échantillon.

$$y_i \sim N\leq ft(\mu, \sigma^2) \; Longrightarrow \; \hat{\mu} \pm (\sigma)/(\sqrt{(n)})\, z_{1-\alpha/2} \tag{1.5.1}$$

L'intervalle de confiance ne dit pas que le paramètre est dans l'intervalle : il dit que la procédure produirait un intervalle le contenant dans 95 % des cas si le modèle était correct. Cette nuance est la source de la plupart des contresens.

Ingénierie des caractéristiques

Avant que les réseaux profonds n'automatisent l'extraction de structure, la conception manuelle des variables était l'essentiel du travail. Elle ne l'est plus devenue, mais la discipline reste nécessaire pour les données tabulaires et pour les variables sensibles au domaine.

Transformation	Objectif	Précaution
Normalisation centrée réduite	comparer des échellessensible aux valeurs extrêmes	
Discrétisation	rendre un modèle linéaire non linéaire	perte d'information, choix des seuils
Indicateurs	encoder une catégorie	une catégorie rare devient un alias vidé
Interaction	capturer un effet conjoint	explosion combinatoire du nombre de termes
Réduction de dimension	supprimer le bruit	interprétabilité perdue
Transformations logarithmiques	réduire l'asymétrie	impossible sur les valeurs négatives

Tableau 1 – Les transformations de variables les plus courantes

Évaluation et validation

Protocole de validation en cinq temps

- **Séparer** en trois jeux : entraînement, validation, test.

2. **Valider croisée** sur le jeu d'entraînement, jamais sur le test.
3. **Choisir les hyperparamètres** sur le jeu de validation.
4. **Consulter le test une seule fois**, à la fin.
5. **Rapporter** l'intervalle de confiance, pas seulement la moyenne.

Le test consulté régulièrement cesse d'être un test : il devient un second jeu d'entraînement, et l'estimation finale est optimiste.

Indicateur	Formule	À utiliser quand
Exactitude	(TP+TN)/(TP+TN+FP+FN)	classes équilibrées
Précision	TP/(TP+FP)	un faux positif coûte cher
Rappel	TP/(TP+FN)	un faux négatif coûte cher
Score F1	moyenne harmonique des deux	compromis entre les deux
Aire sous la courbe	aire du tracé rappel-précision	comparaison de classifieurs tous seuples
Matrice de confusion	tableau des quatre cas	audit, jamais comme métrique unique

Tableau 2 – Indicateurs de classification et ce qu'ils mesurent

Le choix de l'indicateur précède celui de l'algorithme : il découle du coût réel de chaque type d'erreur.

A retenir

- La préparation des données précède le choix du modèle et détermine le plafond de performance.
- Un intervalle de confiance porte sur la procédure, pas sur le paramètre estimé.
- Le test consulté régulièrement cesse d'être un test.
- Le choix de l'indicateur découle du coût de chaque type d'erreur, pas de la facilité de calcul.

Chapitre 1.6

Les disciplines qui préparent l'IA de demain

Cinq domaines dont la rencontre déterminera les dix prochaines années.

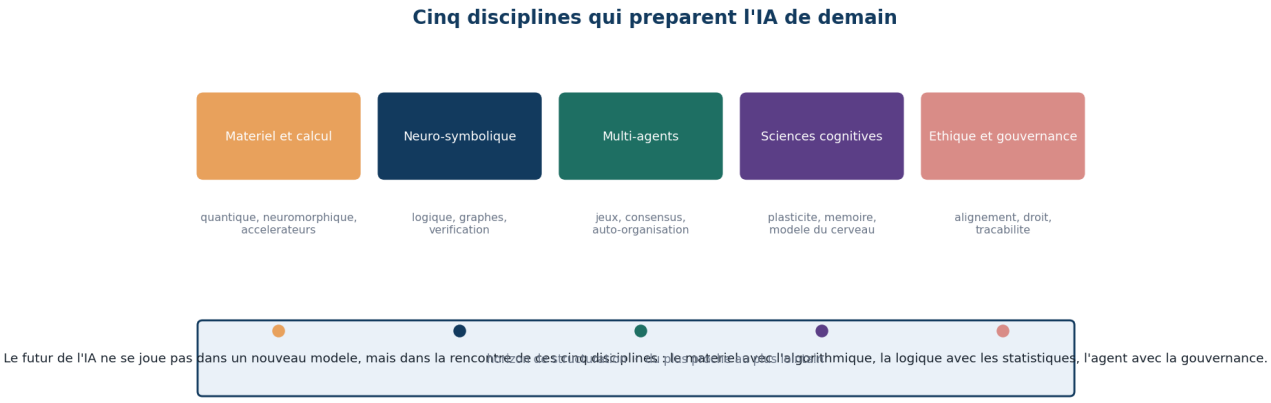


Figure 1 – Cinq disciplines qui préparent l'IA de demain

Discipline	Question qu'elle adresse	Ce qu'elle éclaireHorizon	
Matériel et calcul	comment calculer plus vite, moins cherle franchissement des murs de coût	court terme	
Neuro-symbolique	comment vérifier un raisonnement	la fin des hallucinations non détectées	moyen terme
Systèmes multi-agents	comment coordonner des agents	l'organisation de travail par essaim	moyen terme
Neurosciences computationnelles	comment apprendre en continu	l'apprentissage permanent, non l'entraînement figé	long terme
Éthique et gouvernance	comment garder le contrôle	la légitimité du système et sa responsabilité	immédiat

Tableau 1 – Les cinq disciplines et ce que chacune éclaire

Les deux dernières lignes ne sont pas de la recherche : elles déterminent ce qui peut être déployé.

Matériel, accélérateurs et informatique quantique

La contrainte matérielle a façonné toute l'histoire de l'intelligence artificielle. La disponibilité simultanée de cartes graphiques abordables et de données massives a déclenché la dernière rupture. L'informatique quantique cherche un accélérateur qualitativement différent, pour des problèmes aujourd'hui intrinsèquement hors de portée.

> **L'apprentissage automatique quantique est un domaine de recherche. Il n'existe à ce jour aucune application industrielle, et la question de savoir si le calcul quantique accélérera un modèle d'apprentissage reste ouverte. Citer un avantage quantique établi sur une tâche concrète serait inexact.**
>
> Précaution de lecture

Architectures neuro-symboliques

L'idée est simple : faire produire une conclusion par un réseau, puis la faire vérifier par un moteur logique. Le réseau apporte la capacité à percevoir des données non structurées ; la logique apporte la garantie qu'une conclusion non soutenue par une règle ne peut pas être prononcée.

- **Vérification de la sortie** : la conclusion produite est soumise à un contrôleur logique ; si aucune règle ne la soutient, elle est rejetée.
- **Exécution de contraintes** : les invariants du domaine sont appliqués avant et après la prédiction, jamais seulement après.

- **Appel d'outils symboliques** : le modèle délègue le calcul exact à un moteur dédié, plutôt que de l'approximer.
- **Explication par trace** : la justification est reconstruite à partir des règles effectivement utilisées.

Pourquoi cette combinaison est prometteuse

Un assistant de requête en langage naturel produit une requête SQL. Le modèle génératif n'a aucune garantie que la requête corresponde à l'intention. En encadrant la génération d'une couche symbolique — typage des colonnes, vérification des prédicats, contrôle des droits d'accès — on obtient un système dont le comportement est borné, et dont l'échec est explicite.

Systèmes multi-agents

L organisations en essaim remplace progressivement l'assistant unique. La théorie des jeux, les protocoles de consensus et l'auto-organisation sont les outils de conception de ces systèmes — et, pour le consensus, la limite théorémique rappelée au premier module s'applique intégralement.

Neurosciences computationnelles et sciences cognitives

Pour construire des systèmes capables d'apprentissage continu plutôt que d'entraînement figé, la recherche s'inspire de la plasticité synaptique et des mécanismes de mémoire du cerveau. Il ne s'agit pas de reproduire le cerveau, mais d'en emprunter des principes dossés : plasticité, sparsité, niveaux de mémoire.

Éthique, alignement et gouvernance

Plus un système est puissant et autonome, plus la question de son contrôle devient critique. L'alignement des objectifs, la théorie du choix social et la cryptographie appliquée à la traçabilité forment le socle technique ; le droit et la gouvernance en forment le cadre opérationnel.

Comment lire une prévision à dix ans

Un document prospectif sur l'intelligence artificielle se lit en séparant trois registres : ce qui est déployé aujourd'hui, ce qui est en recherche expérimentale, et ce qui relève de la spéculation. Ces trois registres se mélangent facilement, et le mélange produit soit une naïveté, soit un déni. Un critère simple : si l'affirmation ne peut pas être testée par une expérimentation publiée, elle relève du troisième registre.

A retenir

- Les cinq disciplines qui comptent ne sont pas cinq voies technologiques, mais cinq questions distinctes.
- Le neuro-symbolique vise la vérifiabilité, pas la performance.
- L'apprentissage quantique reste un domaine de recherche sans application industrielle.
- Trois registres de lecture — déployé, recherché, spéculatif — doivent être séparés dans tout document prospectif.

Abréviations, notation et ressources de consultation

Annexes

Annexes

Ce cours se termine par deux annexes de référence. La première donne la signification de chaque sigle employé, sous les deux formes — sigle et développé — car la litterature emploie indifféremment l'une ou l'autre. La seconde donne la signification de chaque symbole mathématique : elle permet de relire une démonstration sans revenir au chapitre dont elle est extraite, et de vérifier qu'un même symbole garde le même sens d'un chapitre à l'autre.

Chapitre A.1

Liste des abréviations

Les sigles ci-dessous sont ceux que le cours emploie. Chacun est donné avec sa forme développée et le sens précis qu'il prend ici, qui n'est pas toujours celui qu'on lui donne ailleurs.

Un sigle employé sans être développé oblige le lecteur à le reconnaître au passage, ou à l'abandonner. Le tableau ci-dessous ferme la liste de ceux que le cours emploie : il est exhaustif pour ce document, et non pour la discipline.

Sigle	Forme développée	Sens dans ce cours
AQ	Assurance qualité	Ensemble des vérifications passées sur un livrable avant sa diffusion.
BL	Base de connaissances	Formalisation en logique d'un domaine de connaissance.
F1	Score F1	Moyenne harmonique de la précision et du rappel, entre 0 et 1.
FN	Faux négatif	Alerte manquée, ou étiquette non attribuée à tort.
FP	Faux positif	Alerte émise à tort, ou étiquette attribuée par erreur.
NN	Vrai négatif	Absence correcte d'alerte, ou refus d attribuer une étiquette.
TP	Vrai positif	Alerte correcte, ou étiquette attribuée à bon escient.
COV	Covariance	Covariance empirique entre deux variables.
H	Entropie	Mesure d'incertitude, en nats si le logarithme est népérien, en bits s'il est binaire.
NC	Nombre de classes	Cardinalité de l'espace des étiquettes.
NP	Non-Polynomial time	Classe de complexité des problèmes dont on ignore l'appartenance à P.
P	Polynomial time	Classe des problèmes solubles en temps polynomial.
PSPACE	Polynomial Space	Classe des problèmes solubles en espace polynomial.
SVM	Support Vector Machine	Machine à vecteurs de support, classifieur à marge maximale.

Tableau 1 – Abréviations employées dans le cours

Chapitre A.2

Table de notation

Les symboles ci-dessous gardent le même sens d'un chapitre à l'autre. Lorsqu'un chapitre doit réemployer un symbole pour un autre objet, il le redéclare sur la page même.

La notation est celle du cours : vecteurs en gras, matrices en majuscules grasses, scalaires en italique. Les espérances sont notées E, les lois de données D, les espaces de paramètres Θ.

Symbole	Signification
$x \in \mathbb{R}^d$	Observation, vecteur de d coordonnées réelles.
$y \in Y$	Étiquette associée, appartenant à l'espace Y.
(x, y)	Exemple annoté.
$f : X \to Y$	Fonction d'apprentissage cible.
h	Prédicteur admis, qui approxime f.
$h^*(x) \in \operatorname{argmin}_h R(h)$	Fonction de Bayes, qui atteint le risque minimal.
$f^*(x) = \operatorname{argmin}_c \mathbb{P}(Y=c \mid X=x)$	Règle de Bayes, qui coïncide avec h^* lorsque le risque est nul sur $h = f^*$.
$L(\hat{y}, y)$	Perte entre la prédiction et l'étiquette.
$R(h) = E[L(h(X), Y)]$	Risque attendu, sous la loi D.
$R(h) = (1)/(m)\sum_{i=1}^{m} L(h(x_i), y_i)$	Risque empirique sur les m exemples observés.
m	Nombre d'exemples d'apprentissage.
d	Dimension de l'espace d'entrée.
h^*	Fonction de Bayes, notation abrégée.
$\Delta = 1 - (R(h))/(R(h^*))$	Marge de généralisation du prédicteur.
$H(Y) = -\sum_c p_c \log p_c$	Entropie de l'étiquette.
$I(X;Y) = H(Y) - H(Y \mid X)$	Information mutuelle entre l'entrée et l'étiquette.
Σ	Matrice de covariance des variables.
$E[X]$	Espérance de la variable aléatoire X.
λ	Marge d'un classifieur, ou valeur propre d'une matrice.
$O(g(n))$	Complexité en temps bornée par g(n).

Tableau 1 – Symboles mathématiques et leur signification

> **Convention**
>
> Toute divergence constatée entre cette table et un chapitre signale une erreur de transcription, non une définition nouvelle : la table fait foi.

> **A retenir**
>
> - Un sigle employé sans être listé ici est un oubli de l'auteur, non une définition nouvelle.
> - Un symbole qui change de sens d'un chapitre à l'autre rend toute démonstration ininterprétable : c'est la principale cause des contresens que nous avons relevés.
> - Ces deux tables sont les seules parties du cours à maintenir à la main : le reste se relit tout seul.

Index des figures et des tableaux

Référence rapide du matériel visuel

Figures

1. Chaîne symbolique et chaîne neuronale : l'articulation neuro-symbolique
1. Descente d'un graphe de calcul à travers les dialectes
1. Temps de calcul selon la taille du problème
1. Le cycle empirique de la science des données

1. Cinq disciplines qui préparent l'IA de demain

Tableaux

1. La hiérarchie de Chomsky et son usage en intelligence artificielle
2. Programmation logique : de la clause a l'inférence
3. Ce que l'impossibilité impose en pratique
1. Programmation fonctionnelle et réseaux de neurones
1. Les dialectes de la chaîne de compilation
2. Les quatre phases de la compilation d'un modèle
1. Les classes de complexité et leur place en intelligence artificielle
2. Les quantités de la théorie de l'information en IA
1. Les transformations de variables les plus courantes
2. Indicateurs de classification et ce qu'ils mesurent
1. Les cinq disciplines et ce que chacune éclaire
1. Abréviations employées dans le cours
1. Symboles mathématiques et leur signification

Fin du document

Intelligence Artificielle Avancée & Federated Learning — Fondations mathématiques de l'apprentissage automatique, algorithmes fédérés, protection de la vie privée, défense contre les attaques, grands modèles de langage fédérés, systèmes embarqués, renforcement appris, applications critiques (santé, V2X), plateforme de référence FedX et feuille de route de recherche.
Ghiloufi Jamel, Architecte Logiciel Principal — Pixel Software Design
Département de Recherche & Développement — Recherche
Pixel Software Design — Édition 2026

© 2026 Pixel Software Design. Tous droits réservés.

Ce document est la propriété exclusive de Pixel Software Design. Toute reproduction, représentation, adaptation ou diffusion, totale ou partielle, par quelque procédé que ce soit et sur quelque support que ce soit, est interdite sans autorisation écrite préalable de l'auteur.

Document généré automatiquement le 29/09/2026 – mise en page Pixel Software Design.